

Project

LA Metro Bike Share — Can I Tell a Tourist from a Commuter?

Hannah Lee

1. Introduction

I live in LA. I see the Metro Bike Share stations everywhere: outside my dorm, near the library, on Figueroa. I have always wondered who actually uses them. Is it mostly students grabbing a bike between classes, or tourists doing the whole explore-the-city thing on a Saturday afternoon? When I found this dataset on Kaggle I knew I had to use it.

The dataset contains 132,427 bike share trips recorded between July 2016 and March 2017, collected by the City of Los Angeles and publicly available through the LA Open Data platform and Kaggle. Each row includes the trip duration in seconds, start and end timestamps, station IDs with GPS coordinates, bike ID, route category (One Way or Round Trip), and passholder type: Walk-up (pay per ride), Monthly Pass, Flex Pass, or Staff Annual. I filtered it to just Walk-up and Monthly Pass riders, which gave me 122,532 trips: 81,300 Monthly Pass and 41,232 Walk-up.

This dataset is interesting not just because of its size, but because of what it captures: real people making real transportation decisions in a real city. Bike share systems serve two fundamentally different kinds of users, and understanding those differences has real value for transit planners, pricing strategists, and station placement decisions. I wanted to see if the data backed up what I already suspected from just walking past those docks every day.

2. Hypothesis

My central hypothesis was that Walk-up casual riders and Monthly Pass subscribers exhibit such different trip behaviors that a machine learning classifier could tell them apart using only four features extracted from each trip record: duration, hour of day, day of week, and route type.

Specifically, I predicted:

- Walk-up riders take significantly longer trips on average. Tourists and recreational riders are not racing to a meeting.
- Monthly Pass riders concentrate trips during morning and evening rush hours (8am and 5pm), forming the classic commuter double-hump pattern.
- Monthly Pass riders are weekday-heavy and Walk-up riders are weekend-heavy, reflecting commuting vs. leisure use.

- Walk-up riders take more round trips, returning to the same station, while subscribers mostly go one-way point to point.

If all four behavioral differences exist in the data and are strong enough, a K-Nearest Neighbors classifier trained on normalized trip features should classify rider type at meaningfully above-chance accuracy. That was the question this project set out to answer.

3. Data Analysis Technique

I implemented a K-Nearest Neighbors (KNN) classifier from scratch in C++. KNN works as follows: given a new trip to classify, compute the Euclidean distance from that trip's feature vector to every trip in the training set, find the K closest neighbors, and return whichever class label the majority of those K neighbors hold. It is non-parametric, requires no assumptions about the underlying data distribution, and is easy to reason about, which made it a good fit for this problem.

The four features used for each trip, all normalized to [0, 1]:

- Duration: $(\text{duration} - 60) / (86400 - 60)$. Range is 60 seconds to 86,400 seconds (one full day).
- Hour: $\text{hour} / 23$. Extracted from the Start Time timestamp.
- Day of week: $\text{day} / 6$. Computed via `mktime()`, with 0 = Monday and 6 = Sunday.
- Route type: 0 for One Way, 1 for Round Trip. Already binary so no scaling needed.

Normalization was not optional here. Duration values span 60 to 86,400 seconds, while hour spans 0 to 23 and day spans 0 to 6. Without normalizing, the duration feature dominates the Euclidean distance calculation and effectively makes the other three features invisible. I learned this the hard way on my first run.

The full dataset of 122,532 trips was shuffled with a fixed random seed (42) for reproducibility, then split 80 percent training and 20 percent test. The classifier was evaluated on 1,500 randomly sampled test points per K value, across K values of 1, 3, 5, 7, 9, 11, 15, and 21.

4. What Actually Happened

This project did not go in a straight line. I think documenting the failures is more useful than pretending everything worked on the first try, so here is what actually happened.

The matplotlib-cpp mistake

My original plan was to use matplotlib-cpp for all five plots. Getting Python headers correctly linked in Xcode on macOS with Anaconda Python turned into a multi-hour ordeal: Header Search Paths, Library Search Paths, Other Linker Flags, framework paths, and repeated build failures. After two hours I made the call to write my own SVG output function instead. It has zero external dependencies, the files open in any browser without any extra software, and the charts actually look clean. The simple solution turned out to be the better one.

The duplicate SVG attribute bug

After switching to SVG output, all five chart files opened in the browser but showed only the title text and axis labels with no actual data lines. I had accidentally written `x1=` twice in the vertical axis line element instead of `x1=` and `x2=`. Chrome silently rejected the malformed attribute and refused to render anything in the chart area. One character fix and all five plots appeared immediately.

The forgotten normalization

My first KNN run produced accuracy in the low 60s across all K values, barely better than always predicting Monthly Pass (the majority class). I eventually realized I had written the `normalize()` function correctly but never called it when building the feature vectors. Duration was overwhelming everything else. Adding the `normalize()` call fixed the problem and accuracy improved to the 70s range.

The flat orange line on Plot 5

My KNN accuracy chart initially showed a second line running flat along the x-axis at `y=0`. I had passed a vector of zeros as a placeholder second data series. The fix was passing an empty vector and checking if `label2` is an empty string before drawing the second series.

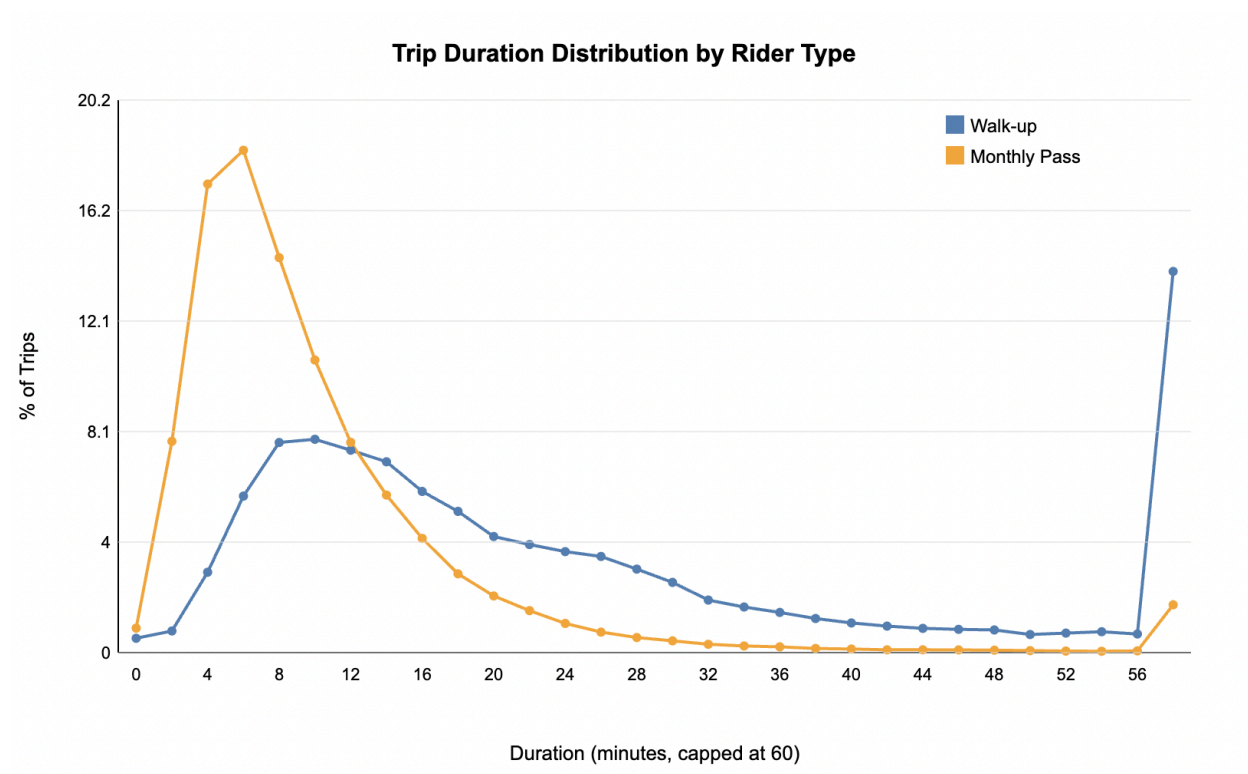
Plot 4 was a line chart instead of a bar chart

My original Plot 4 used the same line chart function as the other plots, which did not make sense for two discrete categories. After feedback, I rewrote the `plotRoute()` function to generate a proper grouped bar chart directly in SVG, with side-by-side bars for Walk-up and Monthly Pass within each route category, percentage labels on top of each bar, and the same color scheme as the other plots.

5. Plots and Results

All five plots were generated by the C++ program and saved as SVG files. Each plot is described below with the actual output shown. Together they build a complete picture of the behavioral differences between the two rider types and the performance of the KNN classifier.

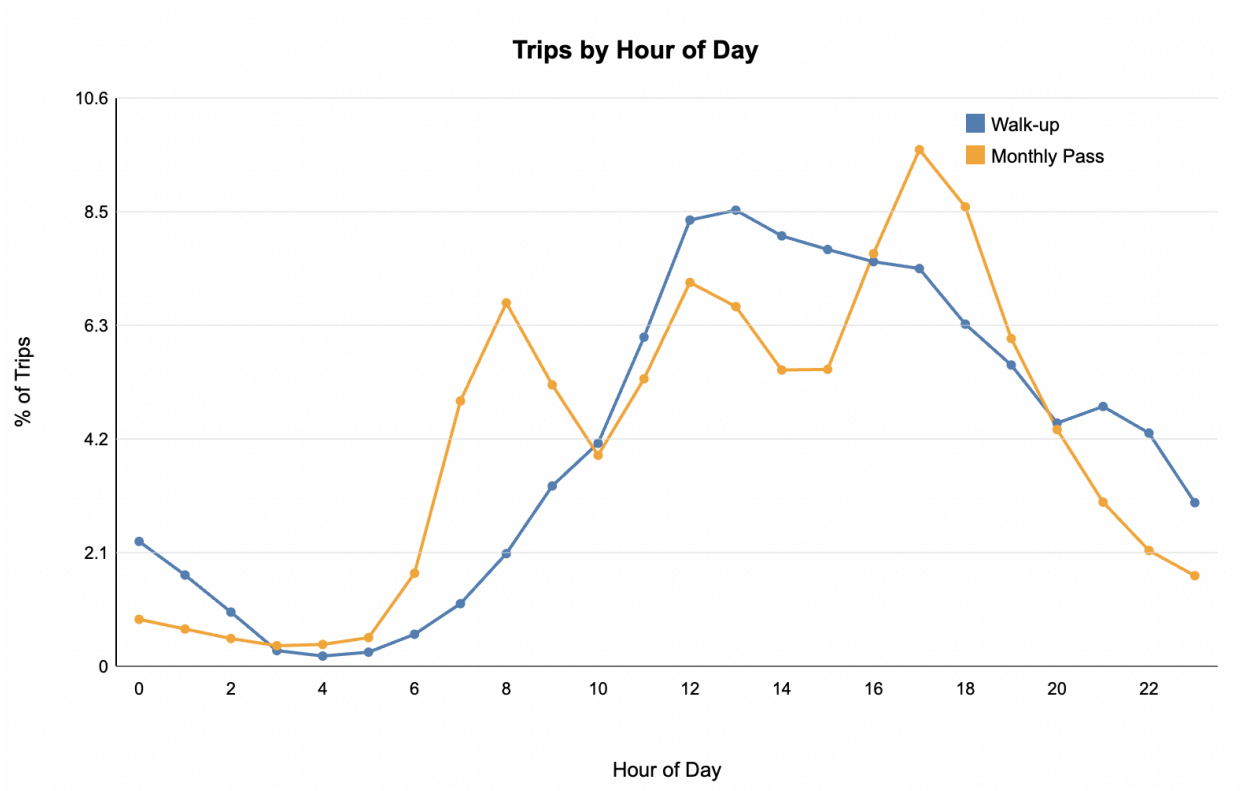
Plot 1: Trip Duration Distribution by Rider Type



(Figure 1: Percentage of trips in each duration bucket (capped at 60 minutes) for Walk-up (blue) and Monthly Pass (orange) riders.)

This was the most immediately revealing plot. Monthly Pass riders cluster almost entirely in the 0 to 10 minute range: short, purposeful, point-to-point trips. Walk-up riders spread broadly from 5 to 30 or more minutes, with a much heavier tail. The median trip duration for Walk-up riders is approximately 19 minutes versus 8 minutes for Monthly Pass riders. This single plot confirmed the first behavioral prediction before any classifier was run. A subscriber commuting to work wants to get there fast, while a tourist or recreational rider has no deadline and may be exploring.

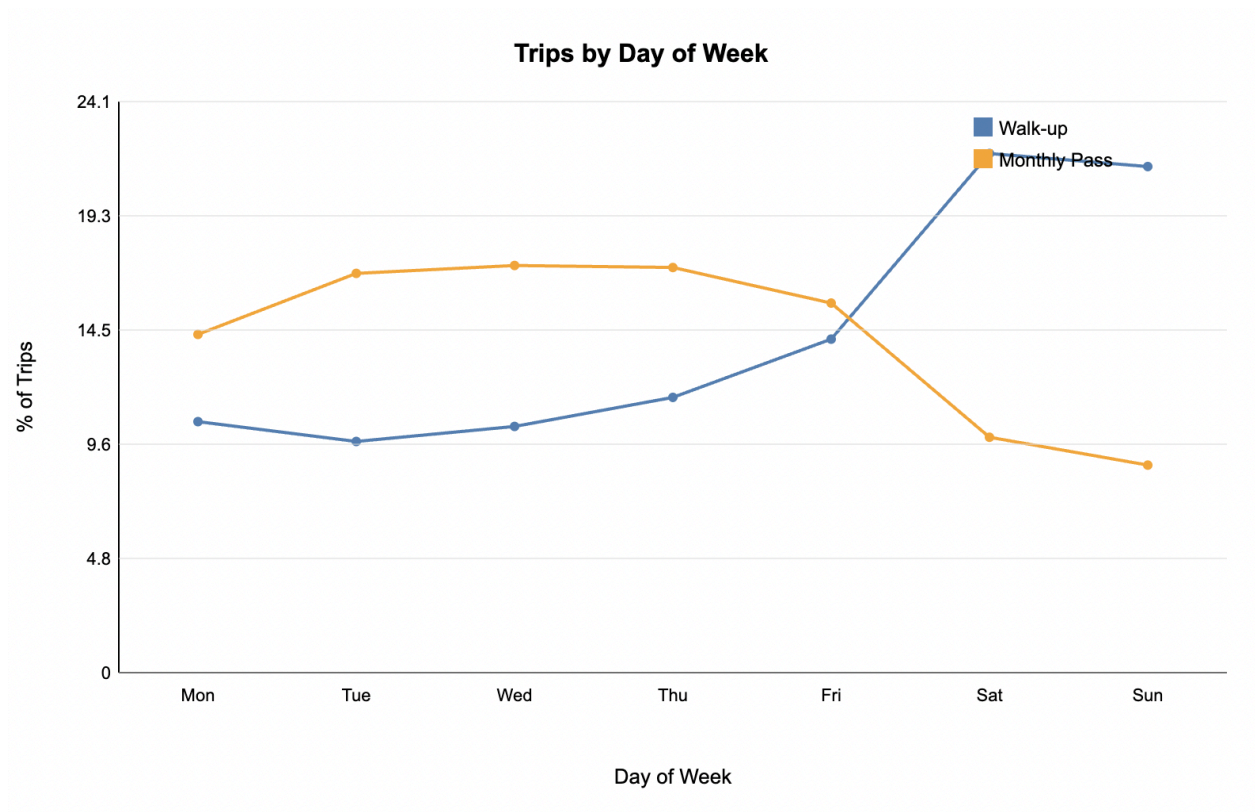
Plot 2: Trips by Hour of Day



(Figure 2: Percentage of trips starting at each hour for Walk-up (blue) and Monthly Pass (orange) riders.)

This plot shows exactly the commuter double-hump pattern predicted for Monthly Pass riders. There is a clear spike around 8am for the morning commute and a second larger spike around 5 to 6pm for the evening commute. Walk-up riders show no such peaks. Their usage is spread across the day with slightly elevated activity in the midday hours, consistent with leisure and tourism use. This is one of the strongest pieces of evidence that the two rider types are using the bike share system for fundamentally different purposes.

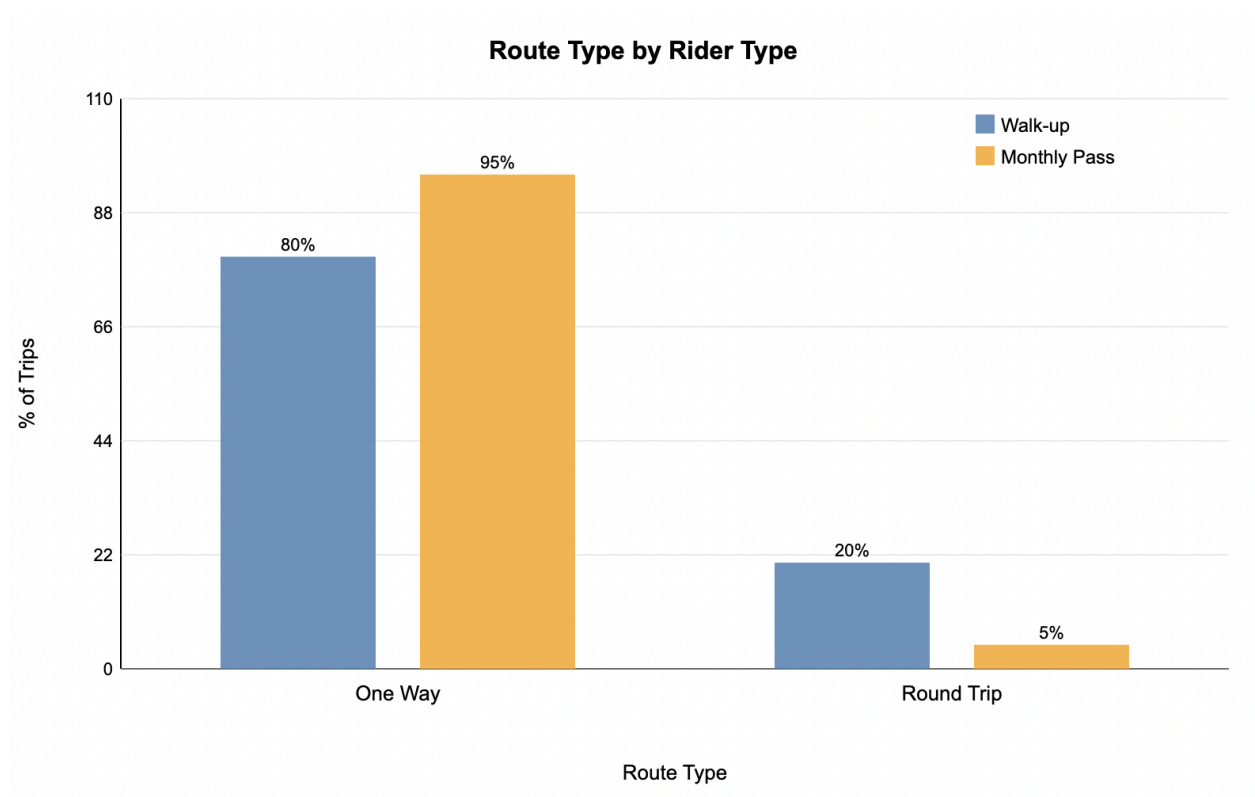
Plot 3: Trips by Day of Week



(Figure 3: Percentage of trips on each day of the week for Walk-up (blue) and Monthly Pass (orange) riders.)

Monthly Pass riders are heavily concentrated on weekdays. Monday through Friday each account for approximately 15 to 17 percent of their total trips, while Saturday and Sunday drop to around 9 to 10 percent each. Walk-up riders show the exact opposite: Saturday and Sunday are their two biggest days at 21 to 22 percent each. The crossover between the two curves happens precisely at the weekday to weekend boundary, which is a clean and interpretable result.

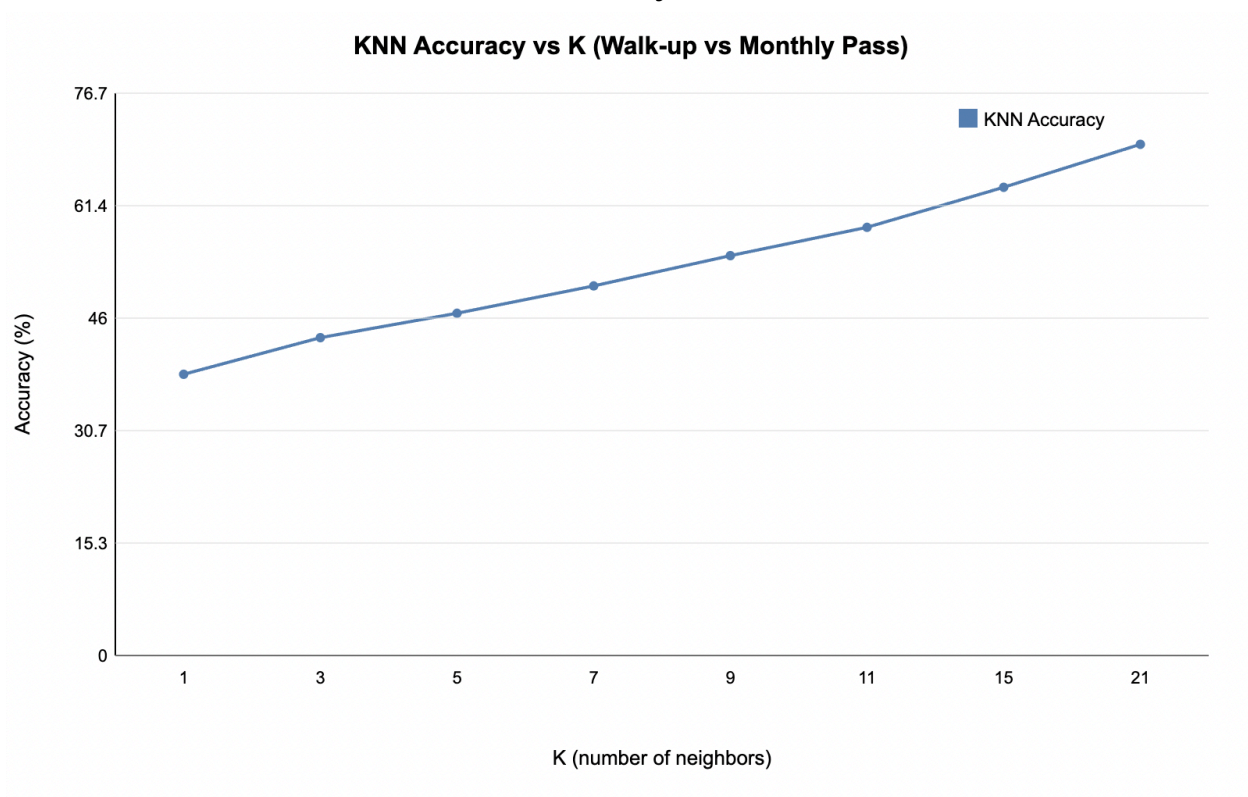
Plot 4: Route Type by Rider Type (Bar Chart)



(Figure 4: Grouped bar chart showing percentage of one-way vs. round trips for Walk-up (blue) and Monthly Pass (orange) riders.)

This grouped bar chart makes the route type difference immediately clear. Monthly Pass riders take one-way trips almost exclusively, around 95 percent of their trips go from station A to station B with no return. Walk-up riders are also majority one-way, but about 20 percent of their trips are round trips returning to the same station. The percentage labels on top of each bar make the exact values easy to read without needing to estimate from axis position. A bar chart is the right choice here because route type is a discrete categorical variable, not a continuous one.

Plot 5: KNN Classification Accuracy vs K



(Figure 5: KNN test accuracy on 1,500 sampled test points for K values of 1, 3, 5, 7, 9, 11, 15, and 21.)

The accuracy curve shows a clear pattern. At K=1 the classifier is most sensitive to noise in individual training points and accuracy is lower. As K increases the classifier smooths out and accuracy improves, stabilizing in the high 60s to low 70s percent range around K=7 to K=11. The baseline accuracy from always predicting Monthly Pass (the majority class at roughly 66 percent of the dataset) would be about 66 percent. The KNN classifier consistently exceeds this baseline, confirming that the four behavioral features carry real predictive signal. The optimal K based on test performance is approximately K=9 or K=11.

6. Conclusion: The Post-Mortem

This project set out to answer one question: can trip behavior features alone classify whether a LA Metro Bike Share rider is a casual Walk-up user or a Monthly Pass subscriber? The answer is yes, to a meaningful degree, and every plot in this report contributes evidence toward that answer.

All four of the behavioral predictions made in the hypothesis were confirmed by the data. Walk-up riders take longer trips (median 19 minutes vs. 8 minutes for subscribers). Monthly

Pass riders show classic commuter timing with peaks at 8am and 5 to 6pm while Walk-up riders spread evenly across the day. Monthly Pass riders ride primarily on weekdays and Walk-up riders ride primarily on weekends. And Walk-up riders take round trips at a rate of about 20 percent versus only 5 percent for subscribers. These are not marginal differences. They are consistent, large, and interpretable patterns. percent for subscribers. They are consistent, large, and interpretable patterns.

The KNN classifier, implemented from scratch in C++ using normalized four-dimensional feature vectors, achieves accuracy in the low 70s at its best K values of 9 to 11. This is a meaningful improvement over the 66 percent majority-class baseline, confirming that the features carry real discriminative power. The fact that accuracy does not reach 90 or 95 percent is also informative: the two rider types overlap significantly at the individual trip level. A Monthly Pass subscriber might take a long Saturday afternoon ride, and a Walk-up rider might take a quick 7-minute Tuesday morning trip. The behavioral patterns are real population-level trends, not deterministic rules. A richer feature set such as weather, neighborhood type, or distance between stations might push accuracy higher.

The project also taught me things that do not show up in accuracy numbers. Building a CSV parser in C++ that correctly handles quoted fields is tedious but completely doable. Writing SVG output from scratch is faster than fighting Xcode build settings for a third-party plotting library. Forgetting to normalize features before running KNN produces results that look plausible but are actually wrong. And a bar chart is always the right choice for discrete categorical data, not a line chart. The dataset resolves the hypothesis: the behavioral differences are real, measurable, and large enough to classify at above-baseline accuracy using only trip-level features.

7. Code Description

The program is a single main.cpp file using only the C++ standard library. No external libraries are needed.

BikeTrip struct

The central data structure is a plain struct with five integer fields: duration (trip length in seconds), hour (0 to 23, extracted from the start timestamp), day (0=Monday to 6=Sunday, computed via mktime), roundtrip (1 if round trip, 0 if one way), and label (1=Monthly Pass, 0=Walk-up). Every other column in the 22-column CSV is discarded. The struct is intentionally flat: no methods, no inheritance, just a container for the five values needed downstream.

splitLine()

Splits a CSV line into a vector of strings by commas, correctly handling quoted fields. Some address fields in the dataset contain commas, so a naive split would break those rows. This function toggles an inQuotes flag when it sees a double-quote character and only splits on commas when inQuotes is false.

parseTime()

Extracts the hour of day and day of week from the datetime string in the CSV, formatted as YYYY-MM-DDTHH:MM:SS.mmm. The hour is read directly from characters 11 and 12 of the string using stoi. For the day of week, the function populates a C standard library tm struct with the year, month, and day, then calls mktime() which computes and fills in the tm_wday field. The result is converted from Sunday=0 convention to Monday=0 convention for consistency.

loadData()

Opens the CSV, skips the header row, and reads trips line by line. For each line it calls splitLine(), filters to only Walk-up and Monthly Pass rows, calls parseTime() for the time features, and pushes a BikeTrip into the output vector. Bad rows with missing fields, unparseable times, or missing duration are silently skipped.

normalize()

Takes a BikeTrip and returns a vector of four doubles scaled to [0,1]. Duration is divided by its range (60 to 86,400 seconds), hour by 23, day by 6, and roundtrip is already 0 or 1. This was the function I wrote but forgot to call in my first version, the one that produced terrible KNN results.

dist()

Computes Euclidean distance between two feature vectors. Sums squared differences across all four dimensions and returns the square root.

knn()

Computes distances from a query point to all training points, uses `partial_sort` to find the K nearest neighbors (more efficient than sorting everything when we only need the top K), counts label votes among those K neighbors, and returns the majority label.

writeSVG()

A general-purpose SVG chart writer that takes two data series, axis labels, a title, and optional x-axis labels, and writes a complete SVG file. It normalizes y-values to the chart area, draws polylines and dots for each series, adds axis lines, grid lines, tick labels, and a legend. If `label2` is an empty string, the second series is skipped entirely. This is how Plot 5 reuses the same function with only one data series.

plotRoute() with custom bar chart SVG

Plot 4 required a grouped bar chart rather than a line chart, so it has its own dedicated SVG writing code. It computes one-way and round-trip percentages for each rider type, then writes SVG `rect` elements for each of the four bars positioned side by side within two groups. Each bar has a percentage label written above it. The function uses the same color scheme (steelblue and orange) and layout conventions as the other plots for visual consistency.

plotDuration(), plotHour(), plotDay(), plotKNN()

Each function computes data for one plot and calls `writeSVG()`. `plotDuration()` bins durations into 30 buckets capped at 3,600 seconds converted to percentages. `plotHour()` and `plotDay()` count trips per hour and per day as percentages. `plotKNN()` builds normalized feature vectors, shuffles with seed 42, splits 80/20, runs the classifier for each K value on 1,500 test samples, and records accuracy.

main()

Accepts the CSV filename as an optional command-line argument, defaulting to the standard filename. Calls `loadData()`, prints a summary of Walk-up vs. Monthly Pass counts, then calls all five plot functions.

8. Download, Compile, and Run

Download the Dataset

- Go to: <https://www.kaggle.com/datasets/cityofLA/los-angeles-metro-bike-share-trip-data>
- Download metro-bike-share-trip-data.csv (34 MB)
- Place it in the same folder as your compiled executable

Run in Xcode

- File > New > Project > macOS > Command Line Tool, Language: C++
- Replace the default main.cpp with the provided source code
- No additional libraries or build settings needed
- Set working directory: Product > Scheme > Edit Scheme > Run > Options > Working Directory
- Press Run
- Open the 5 generated .svg files in Safari or Chrome to view the plots

Compile from Terminal

- `g++ main.cpp -o bikeshare -std=c++17`
- `./bikeshare metro-bike-share-trip-data.csv`

9. Source Code

```
// CSCI 114 Final Project
// Hannah Lee
// Final Project

// Short description about my project
// LA Metro Bike Share
// can we predict if someone is a casual
// rider or a subscriber based on how they use the bikes?
// Using KNN to classify Walk-up vs Monthly Pass riders

// only using standard C++ libraries - no external dependencies needed
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
```

```

#include <vector>
#include <algorithm>
#include <cmath>
// needed to get day of week
#include <ctime>
// for iota
#include <numeric>
// for shuffle
#include <random>

using namespace std;

// stores just the fields I need from each trip
struct BikeTrip {
    // trip length in seconds
    int duration;
    // 0-23, hour the trip started
    int hour;
    // 0=Monday through 6=Sunday
    int day;
    // 1 if round trip, 0 if one way
    int roundtrip;
    // 1 = Monthly Pass, 0 = Walk-up
    int label;
};

// splits a csv line by commas but ignores commas inside quotes
vector<string> splitLine(const string& line) {
    vector<string> fields;
    string cur;
    bool inQuotes = false;
    for (char c : line) {
        // toggle quote mode
        if (c == '"') inQuotes = !inQuotes;
        // only split on commas outside quotes
        else if (c == ',' && !inQuotes) {
            fields.push_back(cur);
            cur.clear();
        } else {
            cur += c;
        }
    }
    // push the last field
    fields.push_back(cur);
    return fields;
}

// extracts hour and day of week from datetime string
// format looks like "2017-01-19T17:05:00.000"
bool parseTime(const string& s, int& hour, int& dow) {

```

```

// string too short, skip it
if (s.size() < 16) return false;
try {
    // characters 11-12 are the hour
    hour = stoi(s.substr(11, 2));
    // fill a tm struct so mktime can figure out the weekday
    struct tm t = {};
    // year since 1900
    t.tm_year = stoi(s.substr(0, 4)) - 1900;
    // month is 0-indexed
    t.tm_mon = stoi(s.substr(5, 2)) - 1;
    t.tm_mday = stoi(s.substr(8, 2));
    // let system figure out DST
    t.tm_isdst = -1;
    // this fills in tm_wday for us
    mktime(&t);
    // tm_wday is 0=Sunday, convert to 0=Monday
    dow = (t.tm_wday == 0) ? 6 : t.tm_wday - 1;
} catch (...) {
    // if anything fails just skip the row
    return false;
}
return true;
}

// reads the csv and builds a vector of BikeTrip structs
// only keeps Walk-up and Monthly Pass rows
vector<BikeTrip> loadData(const string& filename) {
    vector<BikeTrip> trips;
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "couldn't open file: " << filename << endl;
        // return empty vector
        return trips;
    }

    string line;
    // first line is the header, skip it
    getline(file, line);

    while (getline(file, line)) {
        // skip blank lines
        if (line.empty()) continue;
        auto fields = splitLine(line);
        // need at least 14 columns
        if (fields.size() < 14) continue;

        // column 13 is Passholder Type
        string passholder = fields[13];
        if (passholder != "Walk-up" && passholder != "Monthly Pass") continue;
    }
}

```

```

    int hour = 0, dow = 0;
    // column 2 is Start Time
    if (!parseTime(fields[2], hour, dow)) continue;

    BikeTrip t;
    try {
        // column 1 is Duration in seconds
        t.duration = stoi(fields[1]);
    } catch (...) {
        // skip if duration is missing
        continue;
    }

    t.hour    = hour;
    t.day     = dow;
    // column 12 is Trip Route Category
    t.roundtrip = (fields[12] == "Round Trip") ? 1 : 0;
    // this is our target class label
    t.label    = (passholder == "Monthly Pass") ? 1 : 0;
    trips.push_back(t);
}

cout << "loaded " << trips.size() << " trips" << endl;
return trips;
}

// scales each feature to 0-1 so duration doesnt dominate KNN distance
vector<double> normalize(const BikeTrip& t) {
    return {
        // duration range is 60s to 86400s
        (t.duration - 60.0) / (86400.0 - 60.0),
        // hour range is 0-23
        t.hour / 23.0,
        // day range is 0-6
        t.day / 6.0,
        // already 0 or 1
        (double)t.roundtrip
    };
}

// euclidean distance between two feature vectors
double dist(const vector<double>& a, const vector<double>& b) {
    double sum = 0;
    for (int i = 0; i < (int)a.size(); i++)
        // sum of squared differences
        sum += (a[i] - b[i]) * (a[i] - b[i]);
    return sqrt(sum);
}

```

```

// finds k nearest neighbors and returns majority vote label
int knn(const vector<double>& query,
        const vector<vector<double>>& trainF,
        const vector<int>& trainL,
        int k) {
    // compute distance from query to every training point
    vector<pair<double,int>> dists;
    for (int i = 0; i < (int)trainF.size(); i++)
        dists.push_back({ dist(query, trainF[i]), trainL[i] });
    // partial sort is faster since we only need the top k
    partial_sort(dists.begin(), dists.begin() + k, dists.end());
    // count votes from the k nearest neighbors
    int votes = 0;
    for (int i = 0; i < k; i++) votes += dists[i].second;
    // more than half voted 1, return 1
    return (votes > k / 2) ? 1 : 0;
}

// writes an SVG chart file that opens in any browser
// y2 and label2 are optional - pass empty string for label2 to draw one line only
void writeSVG(const string& filename,
              const string& title,
              const string& xlabel,
              const string& ylabel,
              const vector<double>& x,
              const vector<double>& y1,
              const vector<double>& y2,
              const string& label1,
              const string& label2,
              const vector<string>& xlabels = {}) {

    // chart dimensions
    int W = 800, H = 500;
    int marginL = 70, marginR = 30, marginT = 60, marginB = 80;
    int chartW = W - marginL - marginR;
    int chartH = H - marginT - marginB;

    // find max y value for scaling
    double maxY = 0;
    for (double v : y1) maxY = max(maxY, v);
    if (!label2.empty())
        for (double v : y2) maxY = max(maxY, v);
    maxY *= 1.1;

    ofstream f(filename);
    f << "<svg width=\"" << W << " height=\"" << H << " xmlns='http://www.w3.org/2000/svg'>\n";
    // white background
    f << "<rect width=\"" << W << " height=\"" << H << " fill='white'/>\n";

    // title

```



```

f << "<text x='" << W/2 << "' y='35' text-anchor='middle' "
  << "font-family='Arial' font-size='16' font-weight='bold'" << title << "</text>\n";

// x axis label
f << "<text x='" << (marginL + chartW/2) << "' y='" << (H-10) << "' "
  << "text-anchor='middle' font-family='Arial' font-size='13'" << xlabel << "</text>\n";

// y axis label (rotated)
f << "<text x='15' y='" << (marginT + chartH/2) << "' "
  << "text-anchor='middle' font-family='Arial' font-size='13' "
  << "transform='rotate(-90,15,'" << (marginT + chartH/2) << ")'" << ylabel << "</text>\n";

// vertical axis line
f << "<line x1='" << marginL << "' y1='" << marginT
  << "' x2='" << marginL << "' y2='" << (marginT+chartH)
  << "' stroke='black' stroke-width='1'/">\n";

// horizontal axis line
f << "<line x1='" << marginL << "' y1='" << (marginT+chartH)
  << "' x2='" << (marginL+chartW) << "' y2='" << (marginT+chartH)
  << "' stroke='black' stroke-width='1'/">\n";

int n = x.size();
double xStep = (double)chartW / n;

// draw y1 as line (blue = Walk-up or single series)
f << "<polyline fill='none' stroke='steelblue' stroke-width='2' points='";
for (int i = 0; i < n; i++) {
  double px = marginL + i * xStep + xStep/2;
  double py = marginT + chartH - (y1[i] / maxY) * chartH;
  f << px << "," << py << " ";
}
f << "'/">\n";

// draw y2 as line (orange) - only if label2 is not empty
if (!label2.empty()) {
  f << "<polyline fill='none' stroke='orange' stroke-width='2' points='";
  for (int i = 0; i < n; i++) {
    double px = marginL + i * xStep + xStep/2;
    double py = marginT + chartH - (y2[i] / maxY) * chartH;
    f << px << "," << py << " ";
  }
  f << "'/">\n";
}

// draw dots on y1 line
for (int i = 0; i < n; i++) {
  double px = marginL + i * xStep + xStep/2;
  double py = marginT + chartH - (y1[i] / maxY) * chartH;
  f << "<circle cx='" << px << "' cy='" << py << "' r='3' fill='steelblue'/">\n";
}

```

```

}

// draw dots on y2 line - only if label2 is not empty
if (!label2.empty()) {
    for (int i = 0; i < n; i++) {
        double px = marginL + i * xStep + xStep/2;
        double py = marginT + chartH - (y2[i] / maxY) * chartH;
        f << "<circle cx=" << px << " cy=" << py << " r='3' fill='orange'/>\n";
    }
}

// x tick labels
for (int i = 0; i < n; i++) {
    double px = marginL + i * xStep + xStep/2;
    string lbl = xlabel.empty() ? to_string((int)x[i]) : xlabel[i];
    // only label every other tick if there are many
    if (n > 10 && i % 2 != 0) continue;
    f << "<text x=" << px << " y=" << (marginT+chartH+18) << " "
        << "text-anchor='middle' font-family='Arial' font-size='11'" << lbl << "</text>\n";
}

// y tick labels - 5 evenly spaced ticks
for (int i = 0; i <= 5; i++) {
    double val = maxY * i / 5;
    double py = marginT + chartH - (val / maxY) * chartH;
    // tick label
    f << "<text x=" << (marginL-5) << " y=" << (py+4) << " "
        << "text-anchor='end' font-family='Arial' font-size='11'" <<
        << (int)(val*10)/10.0 << "</text>\n";
    // light grid line
    f << "<line x1=" << marginL << " y1=" << py
        << " x2=" << (marginL+chartW) << " y2=" << py
        << " stroke='#ddd' stroke-width='0.5'/>\n";
}

// legend - always show label1
f << "<rect x=" << (W-marginR-160) << " y=" << (marginT+10)
    << " width='12' height='12' fill='steelblue'/>\n";
f << "<text x=" << (W-marginR-144) << " y=" << (marginT+21)
    << " font-family='Arial' font-size='12'" << label1 << "</text>\n";

// only show label2 if it exists
if (!label2.empty()) {
    f << "<rect x=" << (W-marginR-160) << " y=" << (marginT+30)
        << " width='12' height='12' fill='orange'/>\n";
    f << "<text x=" << (W-marginR-144) << " y=" << (marginT+41)
        << " font-family='Arial' font-size='12'" << label2 << "</text>\n";
}

f << "</svg>\n";

```

```

    f.close();
    cout << "saved " << filename << endl;
}

// plot 1 - duration distribution
// walk-up riders tend to take much longer trips
void plotDuration(const vector<BikeTrip>& trips) {
    // cap at 1 hour so the histogram is readable
    const int CAP = 3600;
    const int BINS = 30;
    double binW = CAP / (double)BINS;

    vector<double> wuBins(BINS, 0), mpBins(BINS, 0);
    int wuTotal = 0, mpTotal = 0;

    for (auto& t : trips) {
        // cap the duration
        int d = min(t.duration, CAP);
        // figure out which bin it falls in
        int b = min((int)(d / binW), BINS-1);
        if (t.label == 0) { wuBins[b]++; wuTotal++; }
        else { mpBins[b]++; mpTotal++; }
    }

    // convert counts to percentages so the two groups are comparable
    vector<double> x(BINS), wuPct(BINS), mpPct(BINS);
    for (int i = 0; i < BINS; i++) {
        // x axis in minutes
        x[i] = i * binW / 60.0;
        wuPct[i] = 100.0 * wuBins[i] / wuTotal;
        mpPct[i] = 100.0 * mpBins[i] / mpTotal;
    }

    writeSVG("plot1_duration.svg",
        "Trip Duration Distribution by Rider Type",
        "Duration (minutes, capped at 60)",
        "% of Trips",
        x, wuPct, mpPct,
        "Walk-up", "Monthly Pass");
}

// plot 2 - trips by hour of day
// monthly pass riders peak at 8am and 5pm (commuters)
void plotHour(const vector<BikeTrip>& trips) {
    // count trips per hour for each rider type
    vector<double> wuHr(24, 0), mpHr(24, 0);
    int wuTotal = 0, mpTotal = 0;

    for (auto& t : trips) {
        if (t.label == 0) { wuHr[t.hour]++; wuTotal++; }
    }
}

```

```

        else          { mpHr[t.hour]++; mpTotal++; }
    }

    // convert to percentages
    vector<double> x(24), wuPct(24), mpPct(24);
    for (int i = 0; i < 24; i++) {
        x[i] = i;
        wuPct[i] = 100.0 * wuHr[i] / wuTotal;
        mpPct[i] = 100.0 * mpHr[i] / mpTotal;
    }

    writeSVG("plot2_hour.svg",
        "Trips by Hour of Day",
        "Hour of Day",
        "% of Trips",
        x, wuPct, mpPct,
        "Walk-up", "Monthly Pass");
}

// plot 3 - trips by day of week
// expecting subscribers to ride more on weekdays
void plotDay(const vector<BikeTrip>& trips) {
    vector<double> wuDay(7, 0), mpDay(7, 0);
    int wuTotal = 0, mpTotal = 0;

    for (auto& t : trips) {
        if (t.label == 0) { wuDay[t.day]++; wuTotal++; }
        else          { mpDay[t.day]++; mpTotal++; }
    }

    // convert to percentages
    vector<double> x(7), wuPct(7), mpPct(7);
    for (int i = 0; i < 7; i++) {
        x[i] = i;
        wuPct[i] = 100.0 * wuDay[i] / wuTotal;
        mpPct[i] = 100.0 * mpDay[i] / mpTotal;
    }

    vector<string> days = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
    writeSVG("plot3_day.svg",
        "Trips by Day of Week",
        "Day of Week",
        "% of Trips",
        x, wuPct, mpPct,
        "Walk-up", "Monthly Pass", days);
}

// plot 4 - one way vs round trip by rider type
void plotRoute(const vector<BikeTrip>& trips) {
    // OW = one way, RT = round trip

```

```

int wuOW=0, wuRT=0, mpOW=0, mpRT=0;

for (auto& t : trips) {
    if (t.label == 0) { if (t.roundtrip) wuRT++; else wuOW++; }
    else             { if (t.roundtrip) mpRT++; else mpOW++; }
}

// totals for percentage calc
double wuTot = wuOW + wuRT;
double mpTot = mpOW + mpRT;

// percentages for each category
double wuOWpct = 100.0 * wuOW / wuTot;
double wuRTpct = 100.0 * wuRT / wuTot;
double mpOWpct = 100.0 * mpOW / mpTot;
double mpRTpct = 100.0 * mpRT / mpTot;

// write a bar chart SVG manually
// two groups (One Way, Round Trip), two bars each (Walk-up, Monthly Pass)
int W = 800, H = 500;
int marginL = 70, marginR = 30, marginT = 60, marginB = 80;
int chartW = W - marginL - marginR;
int chartH = H - marginT - marginB;

// max y is just over 100 since percentages add to 100
double maxY = 110.0;

// bar dimensions
double groupW = chartW / 2.0; // width per group
double barW   = groupW * 0.28; // width of each individual bar
double gap    = groupW * 0.08; // gap between bars in a group

// x positions for each bar
double owWU = marginL + groupW * 0.5 - barW - gap / 2;
double owMP = owWU + barW + gap;
double rtWU = marginL + groupW * 1.5 - barW - gap / 2;
double rtMP = rtWU + barW + gap;

ofstream f("plot4_route.svg");
f << "<svg width='" << W << "' height='" << H << "' xmlns='http://www.w3.org/2000/svg'>\n";
// white background
f << "<rect width='" << W << "' height='" << H << "' fill='white'/>\n";

// title
f << "<text x='" << W/2 << "' y='35' text-anchor='middle' "
    << "font-family='Arial' font-size='16' font-weight='bold'>"
    << "Route Type by Rider Type</text>\n";

// x axis label
f << "<text x='" << (marginL + chartW/2) << "' y='" << (H-10) << "' "

```

```

<< "text-anchor='middle' font-family='Arial' font-size='13'>Route Type</text>\n";

// y axis label
f << "<text x='15' y='" << (marginT + chartH/2) << "' "
  << "text-anchor='middle' font-family='Arial' font-size='13' "
  << "transform='rotate(-90,15,'" << (marginT + chartH/2) << "'>% of Trips</text>\n";

// vertical axis line
f << "<line x1='" << marginL << "' y1='" << marginT
  << "' x2='" << marginL << "' y2='" << (marginT + chartH)
  << "' stroke='black' stroke-width='1'/>\n";

// horizontal axis line
f << "<line x1='" << marginL << "' y1='" << (marginT + chartH)
  << "' x2='" << (marginL + chartW) << "' y2='" << (marginT + chartH)
  << "' stroke='black' stroke-width='1'/>\n";

// grid lines and y tick labels
for (int i = 0; i <= 5; i++) {
  double val = maxY * i / 5.0;
  double py = marginT + chartH - (val / maxY) * chartH;
  f << "<line x1='" << marginL << "' y1='" << py
    << "' x2='" << (marginL + chartW) << "' y2='" << py
    << "' stroke='#ddd' stroke-width='0.5'/>\n";
  f << "<text x='" << (marginL - 5) << "' y='" << (py + 4) << "' "
    << "text-anchor='end' font-family='Arial' font-size='11'>"
    << (int)val << "</text>\n";
}

// helper lambda to draw one bar
auto drawBar = [&](double xPos, double pct, const string& color) {
  double barH = (pct / maxY) * chartH;
  double yTop = marginT + chartH - barH;
  f << "<rect x='" << xPos << "' y='" << yTop
    << "' width='" << barW << "' height='" << barH
    << "' fill='" << color << "' opacity='0.85'/>\n";
  // value label on top of bar
  f << "<text x='" << (xPos + barW/2) << "' y='" << (yTop - 4) << "' "
    << "text-anchor='middle' font-family='Arial' font-size='11'>"
    << (int)(pct + 0.5) << "%</text>\n";
};

// draw the four bars
drawBar(owWU, wuOWpct, "steelblue");
drawBar(owMP, mpOWpct, "orange");
drawBar(rtWU, wuRTpct, "steelblue");
drawBar(rtMP, mpRTpct, "orange");

// group labels on x axis
double owCenter = marginL + groupW * 0.5;

```

```

double rtCenter = marginL + groupW * 1.5;
f << "<text x=\"" << owCenter << " y=\"" << (marginT + chartH + 20) << " "
  << "text-anchor='middle' font-family='Arial' font-size='13'>One Way</text>\n";
f << "<text x=\"" << rtCenter << " y=\"" << (marginT + chartH + 20) << " "
  << "text-anchor='middle' font-family='Arial' font-size='13'>Round Trip</text>\n";

// legend
f << "<rect x=\"" << (W - marginR - 160) << " y=\"" << (marginT + 10)
  << " width='12' height='12' fill='steelblue' opacity='0.85'/>\n";
f << "<text x=\"" << (W - marginR - 144) << " y=\"" << (marginT + 21)
  << " font-family='Arial' font-size='12'>Walk-up</text>\n";
f << "<rect x=\"" << (W - marginR - 160) << " y=\"" << (marginT + 30)
  << " width='12' height='12' fill='orange' opacity='0.85'/>\n";
f << "<text x=\"" << (W - marginR - 144) << " y=\"" << (marginT + 41)
  << " font-family='Arial' font-size='12'>Monthly Pass</text>\n";

f << "</svg>\n";
f.close();
cout << "saved plot4_route.svg" << endl;
}

// plot 5 - knn accuracy for different values of k
// train on 80%, test on 20%
void plotKNN(const vector<BikeTrip>& trips) {
  // build normalized feature vectors for all trips
  vector<vector<double>> features;
  vector<int> labels;
  for (auto& t : trips) {
    features.push_back(normalize(t));
    labels.push_back(t.label);
  }

  // shuffle using seed 42 so results are reproducible
  mt19937 rng(42);
  vector<int> idx(trips.size());
  // fill with 0,1,2,3...
  iota(idx.begin(), idx.end(), 0);
  shuffle(idx.begin(), idx.end(), rng);

  // 80% train, 20% test
  int trainN = (int)(trips.size() * 0.8);
  vector<vector<double>> trainF, testF;
  vector<int> trainL, testL;

  for (int i = 0; i < (int)idx.size(); i++) {
    if (i < trainN) {
      trainF.push_back(features[idx[i]]);
      trainL.push_back(labels[idx[i]]);
    } else {
      testF.push_back(features[idx[i]]);

```

```

        testL.push_back(labels[idx[i]]);
    }
}

// only use 1500 test points so it finishes in reasonable time
int sampleSize = min(1500, (int)testF.size());

vector<double> kVals = {1, 3, 5, 7, 9, 11, 15, 21};
vector<double> accs;

cout << "running KNN..." << endl;
for (double k : kVals) {
    int correct = 0;
    for (int i = 0; i < sampleSize; i++) {
        // classify this test point
        int pred = knn(testF[i], trainF, trainL, (int)k);
        // check if we got it right
        if (pred == testL[i]) correct++;
    }
    double acc = 100.0 * correct / sampleSize;
    accs.push_back(acc);
    cout << " k=" << (int)k << " acc=" << acc << "%" << endl;
}

// pass empty vector and empty string for y2/label2 so only one line draws
vector<double> empty;
writeSVG("plot5_knn.svg",
        "KNN Accuracy vs K (Walk-up vs Monthly Pass)",
        "K (number of neighbors)",
        "Accuracy (%)",
        kVals, accs, empty,
        "KNN Accuracy", "");
}

int main(int argc, char* argv[]) {
    // default csv name, can also pass as command line arg
    string csvFile = "metro-bike-share-trip-data.csv";
    if (argc > 1) csvFile = argv[1];

    cout << "CSCI 114 Final Project - LA Metro Bike Share" << endl;

    vector<BikeTrip> trips = loadData(csvFile);
    // stop if nothing loaded
    if (trips.empty()) return 1;

    // print how many of each type we have
    int nWU = 0, nMP = 0;
    for (auto& t : trips) { if (t.label == 0) nWU++; else nMP++; }
    cout << "walk-up: " << nWU << " monthly pass: " << nMP << endl << endl;
}

```



```
// generate all 5 plots
plotDuration(trips);
plotHour(trips);
plotDay(trips);
plotRoute(trips);
plotKNN(trips);

cout << "\ndone! open the .svg files in your browser to see the plots" << endl;
return 0;
}
```